- Query/compute layer (Flink/batch jobs)
- Apache Paimon tables (metadata and data files)
- Storage layer (HDFS/S3/GCS)

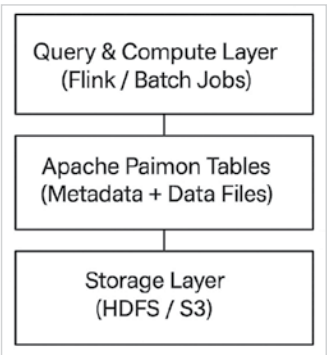

Figure 1: Paimon architecture

Key components are table metadata, data files, snapshots, and compaction which maintains performance by merging small files. This architecture allows Paimon to support streaming writes and batch reads efficiently.

Setting up Apache Paimon on Linux

Apache Paimon runs smoothly on Linux and integrates easily with existing big data stacks.

For running Paimon, the prerequisites are:

- Linux-based system (Ubuntu/CentOS)
- Java 8 or later
- Apache Flink (standalone or cluster mode)

Basic setup steps are:

- Download Apache Flink and Apache Paimon binaries.
- Configure Flink to include the Paimon connector JAR.
- Set up a warehouse directory on HDFS or local file system.

Example configuration in *flink-conf.yaml:*
table.store.path: file:///data/paimon/warehouse

```
vikas-ubuntu-vm mkdir -p/data/paimon/warehouse
vikas-ubuntu-vm cd /lib
vikas-ubuntu-/1ib
flink-1.14.4-bin-scalla_2.12.tgz
paimon-flink-0.6.1.jar
```

Figure 2: Paimon setup

Basic data operations with Apache Paimon

Apache Paimon tables can be created using SQL via Flink:

```
CREATE TABLE orders (
order_id BIGINT,
 customer_id BIGINT,
 amount DOUBLE,
 order_time TIMESTAMP(3),
 PRIMARY KEY (order_id) NOT ENFORCED
) WITH (
 'connector' = 'paimon',
 'path' = 'file:///data/paimon/warehouse/orders'
);
```

This creates a primary-key table, allowing updates and deletes.

```
Flink SQL CLI /v1.14.4, 14d1951 @ 2022-01-28
19:58:35, Scala 2.12.15, Java 1.8.0_292, Unix
Driven Flink Interactive JDBC

Flink SQL> CREATE TABLE orders (
    order_id BIGINT,
    customer_id BIGNT,
    amount DOUBLE,
    order_time TIMESCAMP(3),
```

Figure 3: Creating a table with Flink

You can insert data as usual:

```
INSERT INTO orders VALUES

      (1, 101, 250.50, CURRENT_TIMESTAMP), (2, 102, 175.00,
CURRENT_TIMESTAMP);
       Updating an existing record is handled automatically
via the primary key:
       INSERT INTO orders VALUES (1, 101, 300.00, CURRENT_
TIMESTAMP);
```

Paimon stores this as a change, not a duplicate record.

Common use cases

Real-time analytics:

- Continuous ingestion from Kafka
- Up-to-date dashboards with minimal latency

Streaming pipelines:

- CDC ingestion from transactional databases
- Unified streaming + batch processing

AI and ML-ready data:

- Incremental feature computation
- Consistent, versioned datasets for training and inference

Apache Paimon's snapshot and incremental-read capabilities make it especially useful for modern AI pipelines. Its open source nature, strong integration with Apache Flink, and support for upserts and incremental processing position it as a powerful choice for modern data platforms. As organisations move towards real-time and AI-driven systems, Apache Paimon provides a solid foundation for building scalable, maintainable, and open data architectures. END

By: Vikas Awasthy

The author is a Bengaluru-based consultant data engineer with 12+ years of experience in designing and delivering enterprise-scale data solutions. His areas of interest include data lakehouses, cloud-native analytics, open source data technologies, and real-time data processing.